

BackEnd-SpringBoot — API REST de E-commerce

- Jairo Angulo
[Repositorio en GitHub](#)

API REST para un sistema básico de e-commerce construida con **Spring Boot 3.1.0**, **Maven** y **PostgreSQL**.

Tecnologías

- Java 17
- Spring Boot 3.1.0 (Web, Data JPA, Validation)
- Spring Security Crypto (BCrypt)
- PostgreSQL
- Lombok
- SpringDoc OpenAPI / Swagger UI

Endpoints

Usuarios (/api/users)

Método	Ruta	Descripción
POST	/api/users/register	Registrar nuevo usuario
POST	/api/users/login	Iniciar sesión
GET	/api/users/{id}	Obtener usuario por ID
PUT	/api/users/{id}	Actualizar usuario
DELETE	/api/users/{id}	Eliminar usuario

Productos (/api/products)

Método	Ruta	Descripción
POST	/api/products	Crear producto
GET	/api/products/{id}	Obtener producto por ID
GET	/api/products	Listar todos los productos
PUT	/api/products/{id}	Actualizar producto
DELETE	/api/products/{id}	Eliminar producto

Recibos (/api/receipts)

Método	Ruta	Descripción
POST	/api/receipts	Crear recibo (calcula total)

Método	Ruta	Descripción
GET	<code>/api/receipts/{id}</code>	Obtener recibo por ID
GET	<code>/api/receipts</code>	Listar todos los recibos
GET	<code>/api/receipts/user/{userId}</code>	Recibos de un usuario
DELETE	<code>/api/receipts/{id}</code>	Eliminar recibo

Ejecución

1. Crear base de datos PostgreSQL llamada `ecommerce`.
2. Configurar credenciales en `application.properties` o variables de entorno (`DB_URL`, `DB_USER`, `DB_PASSWORD`).
3. Ejecutar:

```
mvn clean spring-boot:run
```

La API corre en `http://localhost:8000`. Swagger disponible en `/swagger-ui.html`.

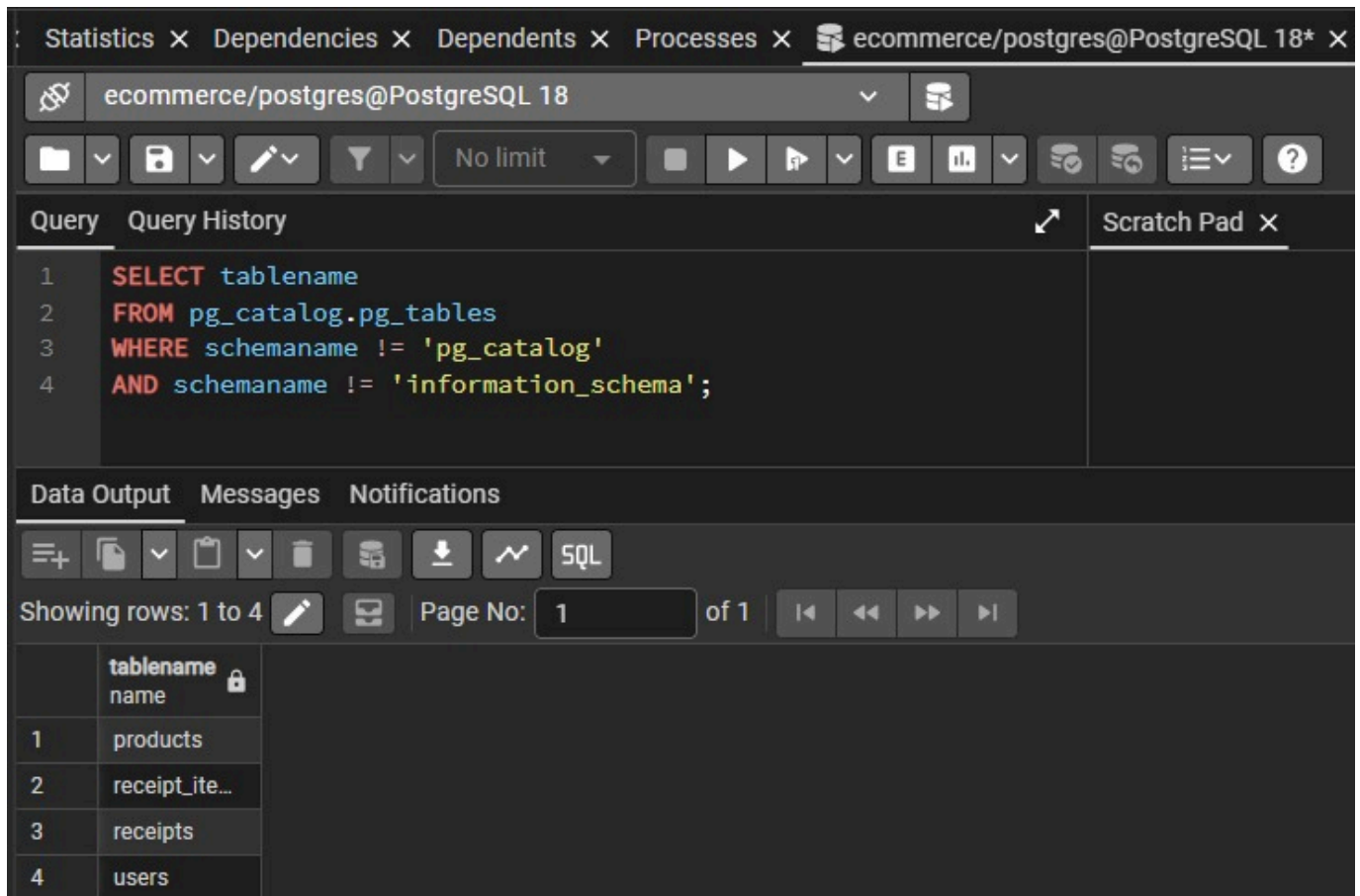
Características principales

- Contraseñas encriptadas con BCrypt (nunca se devuelven en respuestas)
- Precios y totales con `BigDecimal` para precisión monetaria
- Cálculo del total de recibos desde la base de datos (no desde el frontend)
- Descuento automático de stock al crear un recibo
- Manejo global de errores con `@RestControllerAdvice`
- Validación de datos de entrada con `jakarta.validation`
- DTOs separados de las entidades JPA
- Arquitectura por capas: controlador → servicio → repositorio

Evidencias

1. Tablas creadas en PostgreSQL

Tablas `users`, `products`, `receipts` y `receipt_items` generadas automáticamente por JPA desde las entidades del proyecto.



2. Ejecución exitosa en consola

Aplicación Spring Boot corriendo correctamente en el puerto 8000 desde Spring Tools.

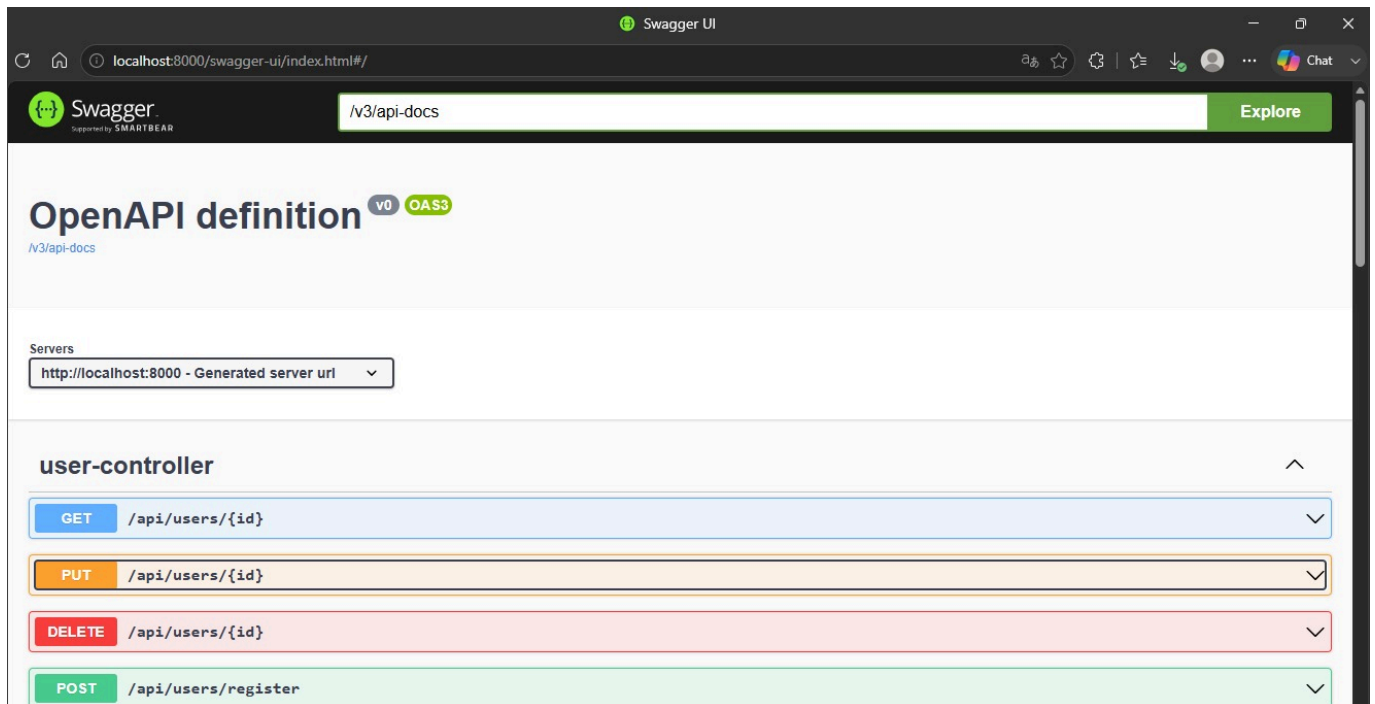
```

2026-07-02T21:36:42.537-05:00 INFO 38232 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8000 (http)
2026-07-02T21:36:42.547-05:00 INFO 38232 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-02T21:36:42.548-05:00 INFO 38232 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.8]
2026-07-02T21:36:42.694-05:00 INFO 38232 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-02T21:36:42.695-05:00 INFO 38232 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1683 ms
2026-07-02T21:36:42.869-05:00 INFO 38232 --- [main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-07-02T21:36:42.941-05:00 INFO 38232 --- [main] org.hibernate.Version : HHH000412: Hibernate ORM core version 6.2.2.Final
2026-07-02T21:36:42.944-05:00 INFO 38232 --- [main] org.hibernate.cfg.Environment : HHH000406: Using bytecode reflection optimizer
2026-07-02T21:36:43.126-05:00 INFO 38232 --- [main] o.h.b.i.BytecodeProviderInitiator : HHH000021: Bytecode provider name : bytebuddy
2026-07-02T21:36:43.311-05:00 INFO 38232 --- [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-07-02T21:36:43.333-05:00 INFO 38232 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-07-02T21:36:43.594-05:00 INFO 38232 --- [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@
2026-07-02T21:36:43.596-05:00 INFO 38232 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-07-02T21:36:43.666-05:00 INFO 38232 --- [main] org.hibernate.orm.dialect : HHH035001: Using dialect: org.hibernate.dialect.PostgreSQLDialect
2026-07-02T21:36:43.977-05:00 INFO 38232 --- [main] o.h.b.i.BytecodeProviderInitiator : HHH000021: Bytecode provider name : bytebuddy
2026-07-02T21:36:45.026-05:00 INFO 38232 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000490: Using JtaPlatform implementation: [org.hibernate.engin
Hibernate:
    alter table if exists receipts
    alter column created_at set data type timestamp(6)
2026-07-02T21:36:45.182-05:00 INFO 38232 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-07-02T21:36:45.832-05:00 WARN 38232 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, databas
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::invokeCleaner has been called by nonapi.io.github.classgraph.utils.FileUtils (file:/C:/Users/USER/.m2/repository/io/github/classgraph/classgraph/4
WARNING: Please consider reporting this to the maintainers of class nonapi.io.github.classgraph.utils.FileUtils
WARNING: sun.misc.Unsafe::invokeCleaner will be removed in a future release
2026-07-02T21:36:46.978-05:00 INFO 38232 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8000 (http) with context path ''
2026-07-02T21:36:46.988-05:00 INFO 38232 --- [main] c.c.e.BackendEcommerceApplication : Started BackendEcommerceApplication in 6.595 seconds (process run
2026-07-02T21:38:21.745-05:00 INFO 38232 --- [nio-8000-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-07-02T21:38:21.751-05:00 INFO 38232 --- [nio-8000-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-02T21:38:21.776-05:00 INFO 38232 --- [nio-8000-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 24 ms

```

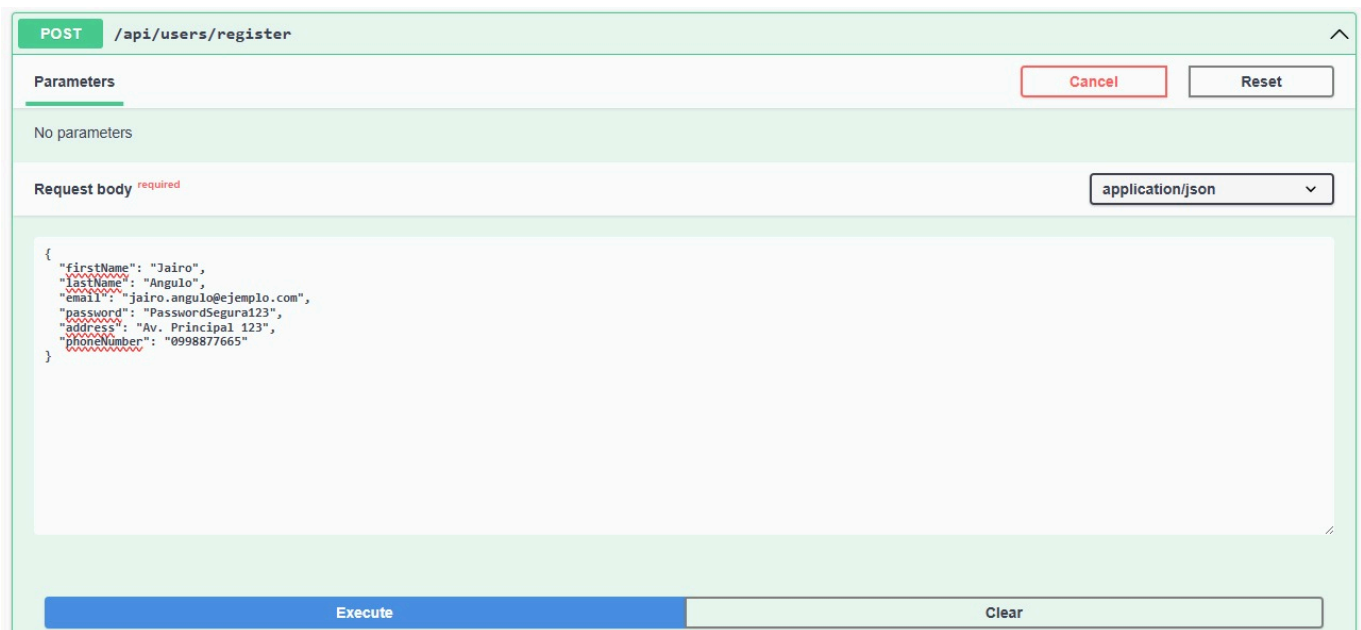
3. Swagger UI — Documentación de endpoints

Interfaz de Swagger con todos los endpoints disponibles: GET, POST, PUT, DELETE para probar la API.



4. Envío de datos desde Swagger a la base de datos

Ejemplo de petición POST con parámetros enviados desde Swagger para crear un registro en la base de datos.



5. Verificación en PostgreSQL

Consulta en PostgreSQL confirmando que los datos se almacenaron correctamente en la base de datos.

Dashboard X Properties X SQL X Statistics X Dependencies X Dependents X Processes X ecommerce

ecommerce/postgres@PostgreSQL 18

Query Query History

1 SELECT * FROM users

Data Output Messages Notifications

Showing rows: 1 to 1

	user_id [PK] integer	address character varying (255)	email character varying (255)	first_name character varying (255)	last_name character varying (255)
1	1	Av. Principal 123	jairo.angulo@ejemplo.c...	Jairo	Angulo

Estructura del proyecto

```
src/main/java/com/coltis/ecommerce
|
|-- BackEndEcommerceApplication.java
|
|-- config
|   |-- PasswordConfig.java
|
|-- controllers
|   |-- ProductController.java
|   |-- ReceiptController.java
|   |-- UserController.java
|
|-- dto
|   |-- ApiError.java
|   |-- CreateReceiptRequest.java
|   |-- LoginRequest.java
|   |-- ProductRequest.java
|   |-- ReceiptItemRequest.java
|   |-- ReceiptItemResponse.java
|   |-- ReceiptResponse.java
|   |-- RegisterUserRequest.java
|   |-- UpdateUserRequest.java
|   |-- UserResponse.java
|
|-- exceptions
|   |-- BadRequestException.java
|   |-- GlobalExceptionHandler.java
|   |-- InvalidCredentialsException.java
|   |-- ResourceNotFoundException.java
|
|-- models
```

```
| |-- Product.java
| |-- Receipt.java
| |-- ReceiptItem.java
| |-- User.java
|
|-- repository
| |-- ProductRepository.java
| |-- ReceiptRepository.java
| |-- UserRepository.java
|
|-- services
| |-- ProductService.java
| |-- ReceiptService.java
| |-- UserService.java
```